

Automatic ICD-10 Category Prediction from Spanish Clinical Literals

Gússem Yahia-Cheikh, María Colomer & Sara Díez

June 1, 2026

1 Introduction

The objective of this project is to automatically assign an ICD-10 category to Spanish clinical literals. ICD codes are standardized medical codes used to represent diagnoses, procedures, symptoms, and other health-related concepts. In this task, the goal is not to predict the complete ICD code, but only its first character, referred to as `y_category`.

For example, if the ICD code is `N801`, the expected category is `N`. If the code is `07CP0ZZ`, the expected category is `0`. Therefore, the task is treated as a single-label multiclass text classification problem, where each input literal must be assigned to one of 36 possible categories: digits from 0 to 9 and letters from A to Z.

The task is challenging because clinical literals are short, noisy, and often contain abbreviations, spelling variation, missing accents, inconsistent capitalization, or mixtures of Spanish and Catalan. In addition, many ICD categories are semantically close, and some literals may be ambiguous without more clinical context.

The main objective of the project is to compare different ways of solving this problem and to analyze which approach is more suitable for this type of data.

2 Dataset Description

Three CSV files were used for the project:

- `codification_data.csv`: labelled clinical literals with their corresponding ICD codes.
- `leaderboard_data.csv`: unlabelled literals for which predictions must be submitted to Kaggle.
- `icd_d_p_pairs.csv`: official ICD code-description pairs.

The main columns used in the project are shown in Table 1.

Column	Description
<code>Literal</code>	Clinical text to classify.
<code>Code</code>	Full ICD code associated with a literal.
<code>Description</code>	Official ICD description.
<code>id</code>	Identifier required for Kaggle submission.
<code>y_category</code>	First character of the ICD code.

Table 1: Main columns used in the project.

The target variable is created using the first character of the ICD code:

$$y_category = \text{first character of Code}$$

This means that the model performs category prediction, not full ICD code prediction.

3 Training Configuration

The main parameters used in the project are shown in Table 2. These values were kept fixed during the improvements in order to compare approaches fairly.

Parameter	Value
Seed	42
Maximum epochs	50
Early stopping patience	10
Batch size	128
Learning rate	2×10^{-5}
Weight decay	0.01
Dropout	0.1
Maximum sequence length	64
Pooling strategy	Mean pooling

Table 2: Training configuration.

The training data is split into training and validation subsets using a stratified split when possible. The validation set is kept as real clinical literals from `codification_data.csv`.

4 Preprocessing

The preprocessing applied to the RoBERTa model was intentionally light. Each literal was converted to a string, leading and trailing spaces were removed, and multiple consecutive spaces were transformed into a single space.

We did not apply any lowercasing, accent, or punctuation removal, the reason behind being that, while removing accents or punctuation, we could also remove useful information, especially in short clinical literals.

5 Technical description of the solution

We did not developed the final solution in a single step. We first explored simple baselines and then progressively moved towards a transformer-based model. The purpose of this process was to understand the difficulty of the task and improve the final submission model.

5.1 Lexical Baseline

The first approach was to develop a lexical baseline based on exact matching. Clinical literals were normalized using lowercasing, accent removal, punctuation cleaning, and whitespace normalization. Then, each normalized literal was matched against previously seen training literals.

If the same literal appeared in the training set, the system predicted the most frequent category associated with that literal. If the literal was not found, the model predicted the majority class.

5.2 Supervised RoBERTa Classifier

The main supervised model was based on the pretrained Spanish biomedical clinical RoBERTa model:

PlanTL-GOB-ES/roberta-base-biomedical-clinical-es

The architecture of the first model we developed consisted of:

- a pretrained RoBERTa encoder,
- a mean pooling operation over token embeddings,
- a dropout layer,
- a linear classification layer.

For each input literal, RoBERTa produced contextual token representations. Since the task requires one prediction per literal, the token embeddings were combined using mean pooling:

$$h = \frac{\sum_{i=1}^n m_i h_i}{\sum_{i=1}^n m_i}$$

where h_i is the hidden representation of token i , and m_i is the attention mask. The attention mask ensures that padding tokens are not included in the average.

The final prediction was computed as:

$$\hat{y} = \text{Linear}(\text{Dropout}(h))$$

We later developed another RoBERTa version, which:

- used an 80/20 train-validation split,
- implemented an early stopping based on validation accuracy,
- generated a valid submission file for *Kaggle*.

This version achieved a validation accuracy of 0.5639 at epoch 13. This became the main baseline for the following experiments.

5.3 Use of the Official ICD Description File

The file `icd_d_p_pairs.csv` contains official ICD codes and their descriptions, and since this file provides additional medical information, we tested whether it could improve the model. A first attempt consisted of adding the official ICD descriptions from `icd_d_p_pairs.csv` as extra training examples. Each description was treated as an additional literal, and the first character of its ICD code was used as the label.

However, this approach reduced the accuracy from approximately 0.564 to 0.54. The most probable reason for this is that since the official ICD descriptions are formal, long and standardized, while the leaderboard examples are short and noisy clinical literals, adding all official descriptions directly to the neural training set introduced examples that did not follow the same distribution as the target data.

Because of this, our final decision was to train the RoBERTa model only on the real supervised literals from `codification_data.csv`.

5.4 Final Improved RoBERTa Version

The final version of our project kept the same main hyperparameters as in subsection 5.2, but improved the training procedure. The main additions were:

- a learning-rate scheduler,
- gradient clipping,
- training accuracy monitoring,
- full-data retraining,
- a three-seed ensemble.

The best validation accuracy was obtained at epoch 10:

$$\text{Validation accuracy}_{p_v5} = 0.5712$$

After selecting epoch 10 as the best epoch, the model was retrained on the full labelled dataset. Three models were trained using seeds 42, 123, and 2026, and their logits were averaged to generate the final submission file:

submission_roberta_mean_v5.csv

6 Results

The main comparison was between the first stable RoBERTa baseline, `p_v3.py`, and the final improved version, `p_v5.py`. The results are shown in Table 3.

File Vers.	Best Epoch	Val. Accuracy	Macro/Weighted F1	Description
<code>p_v3.py</code>	13	0.5639	0.50 / 0.55	Stable RoBERTa baseline trained on an 80/20 train-validation split.
<code>p_v5.py</code>	10	0.5712	0.49 / 0.54	Final improved RoBERTa model used to generate the final submission.

Table 3: Comparison between the two main RoBERTa versions.

The best validation accuracy was obtained by `p_v5.py`, the model correctly predicting approximately 57.12% of the validation examples.

The detailed classification report for the best validation model is shown in Table 4.

Table 4: Classification report for the best RoBERTa model on the validation set.

Class	Precision	Recall	F1-score	Support
0	0.63	0.72	0.67	228
1	0.64	0.56	0.60	57
2	0.31	0.10	0.15	81
3	0.52	0.43	0.47	118
4	0.47	0.31	0.38	80
5	0.45	0.26	0.33	82
6	0.38	0.23	0.29	128
7	0.32	0.09	0.14	79
8	0.52	0.40	0.45	81
9	0.42	0.32	0.36	56
A	0.00	0.00	0.00	4
B	0.69	0.84	0.76	116
C	0.77	0.73	0.75	41
D	0.51	0.64	0.57	67
E	0.47	0.68	0.55	100
F	0.56	0.66	0.61	61
G	0.43	0.60	0.50	43
H	0.52	0.76	0.62	29
I	0.62	0.81	0.70	74
J	0.49	0.55	0.52	47
K	0.54	0.74	0.63	76

Class	Precision	Recall	F1-score	Support
L	0.44	0.50	0.47	22
M	0.66	0.81	0.73	58
N	0.53	0.71	0.61	107
O	0.56	0.57	0.57	301
P	0.80	0.80	0.80	20
Q	0.40	0.48	0.44	29
R	0.61	0.75	0.68	72
S	0.79	0.79	0.79	14
T	0.75	0.21	0.33	14
U	1.00	0.75	0.86	4
V	0.38	0.09	0.15	98
W	0.00	0.00	0.00	1
X	0.00	0.00	0.00	2
Y	0.50	0.29	0.36	7
Z	0.68	0.84	0.75	343
Accuracy	—	—	0.57	2740
Macro average	0.51	0.50	0.49	2740
Weighted average	0.55	0.57	0.54	2740

7 Analysis and Discussion

The final model improved the validation accuracy from 0.5639 to 0.5712. Although the improvement is small, it is meaningful because both models use the same pretrained RoBERTa backbone and the same main hyperparameters. The gain mainly comes from improving the training procedure rather than changing the model architecture.

The training curves show signs of overfitting. In `p_v5.py`, the best validation accuracy was obtained at epoch 10. After this point, training accuracy continued increasing, but validation accuracy decreased. For example, by epoch 20, the training accuracy had increased to 0.7974, while validation accuracy had dropped to 0.5420. This means that the model was learning the training examples more specifically, but it was not generalizing better to unseen validation data.

The classification report shows that performance is strongly affected by class imbalance. Frequent categories such as Z, O, O, B, and I achieved stronger results. In contrast, rare categories such as A, W, and X obtained an F1-score of 0.00 because the model had very few examples from which to learn them.

The macro F1-score was lower than the accuracy, which confirms this imbalance problem. Accuracy is dominated by frequent classes, while macro F1 gives equal importance to every class. Therefore, the model may seem acceptable from the accuracy value, but it still struggles with rare categories.

The experiment with official ICD descriptions also showed an important limitation. Although these descriptions contain useful medical information, adding them directly as training examples reduced performance. This suggests that their formal and standardized style does not match the short, noisy style of the real clinical literals. For this reason, the official ICD descriptions would probably be more useful as retrieval support than as direct neural training data.

Finally, the full-data retraining stage was used only for the final submission. Since all labelled examples were used for training at that stage, the printed full-training accuracies are not validation accuracies. The real performance of the final ensemble can only be measured through the Kaggle leaderboard.

8 Final Conclusions

In this project, we addressed the task of automatic ICD-10 category prediction from Spanish clinical literals. The task was formulated as a single-label multiclass classification problem, where the target label is the first character of the full ICD code.

The development process started with classical baselines, continued with a first RoBERTa model, tested the use of official ICD descriptions, and ended with an improved RoBERTa training strategy. The best validation result was obtained by the final improved RoBERTa model, with a validation accuracy of 0.5712.

The results show that biomedical RoBERTa is more suitable for this task than simple lexical or retrieval-based methods. However, the task remains difficult because the literals are short, noisy, abbreviated, and affected by class imbalance.